

CORE COMPUTER SCIENCE

Operating Systems

A visual reference on processes, scheduling, synchronization, memory & disks

- › Process & Thread Basics
- › CPU Scheduling Algorithms
- › Critical Section & Sync
- › Deadlocks & Banker's Algorithm
- › Memory Management & Paging
- › Page Replacement Algorithms
- › Disk Scheduling
- › Key Terms & Definitions

01 Introduction to Operating Systems

An **Operating System (OS)** is an interface between the user and hardware. It is responsible for the execution of all processes, resource allocation, CPU management, file management, and many other tasks. The purpose of an OS is to provide an environment in which a user can execute programs conveniently and efficiently.

Types of Operating Systems

1. Batch OS

Similar jobs are grouped and stored in main memory. A job is assigned to the CPU only after the previous job finishes.

2. Multiprogramming OS

Multiple jobs sit in memory; when one waits on I/O, the OS switches to another so the CPU is never idle.

3. Multitasking OS

Combines multiprogramming with fast CPU switching so the user can interact with each program as it runs.

4. Time Sharing OS

Requires ongoing user interaction (usually via keyboard); the OS responds with output for each instruction.

5. Real Time OS

Built for dedicated systems that must complete a fixed set of tasks within strict deadlines.

02 Process & Threads

A **process** is a program under execution. The **program counter (PC)** holds the address of the next instruction to execute. Every process is represented in the OS by a **Process Control Block (PCB)**.

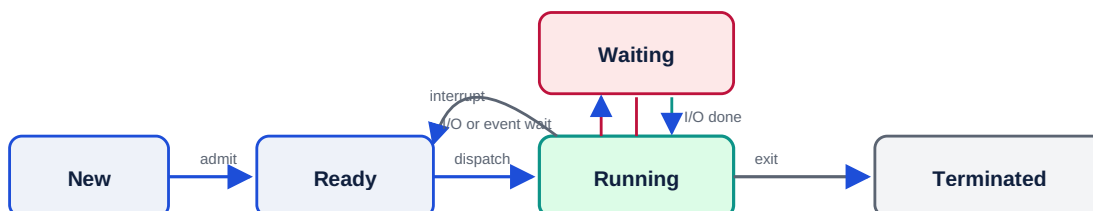


Fig 1 — Process life-cycle: a process moves between New, Ready, Running, Waiting and Terminated states.

Process Scheduling Terms

Term	Meaning
Arrival Time	Time at which the process arrives in the ready queue.
Completion Time	Time at which the process finishes execution.
Burst Time	Time required by the process for CPU execution.
Turn Around Time (TAT)	Completion Time – Arrival Time
Waiting Time (WT)	Turn Around Time – Burst Time

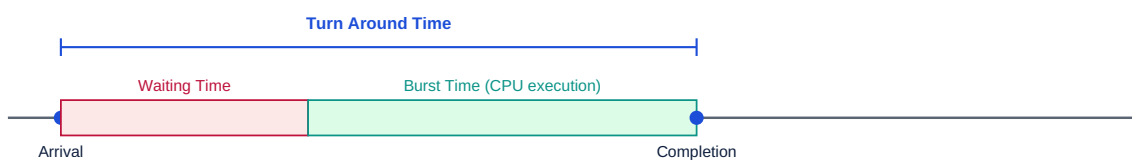


Fig 2 — Relationship between Arrival, Waiting, Burst and Turn Around Time.

Threads IMPORTANT

A **thread** is a lightweight process — the basic unit of CPU utilization. A process can perform multiple tasks at once by running several threads. Each thread has its own program counter, register set, and stack, but shares the code section, data section, files and signals with other threads of the same process.

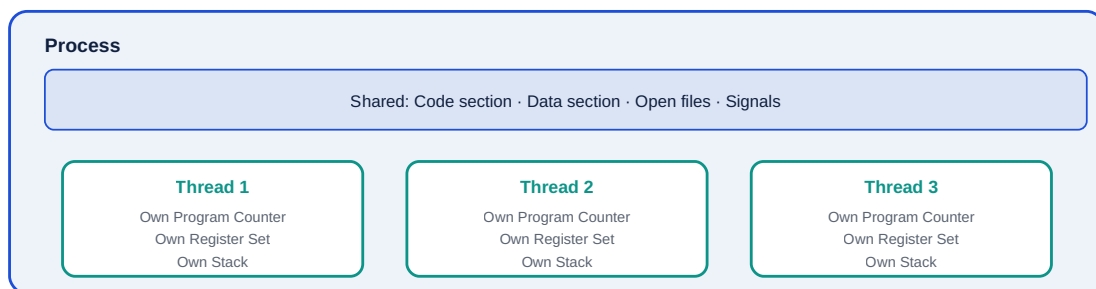


Fig 3 — Multiple threads inside one process share code, data and files, but keep private PC, registers and stack.

NOTE

A new thread, or a child process of a given process, can be created using the **fork()** system call. A process with n **fork()** calls generates $2^n - 1$ child processes.

There are two types of threads: **User threads** — implemented by the user, and **Kernel threads** — implemented by the OS.

03 CPU Scheduling Algorithms

Algorithm	How it works
FCFS	Simplest algorithm — schedules processes strictly by arrival time.
SJF	Process with the shortest burst time is scheduled first (non-preemptive).
SRTF	Preemptive version of SJF — always runs the job with shortest remaining time.
Round Robin	Each process gets a fixed time quantum in cyclic order.
Priority (Non-preemptive)	Highest priority runs first; ties are broken by arrival time.
HRRN	Schedules the process with the highest response ratio; avoids starvation.
MLQ	Processes sit in separate priority queues; higher queues fully drain before lower ones run.
MLFQ	Like MLQ, but a process can move between queues based on its CPU-burst behaviour.

$$\text{Response Ratio (HRRN)} = (\text{Waiting Time} + \text{Burst Time}) / \text{Burst Time}$$

04 The Critical Section Problem

- **Critical Section** — the part of the code where shared variables are accessed or updated.
- **Remainder Section** — the rest of the program, excluding the critical section.
- **Race Around Condition** — output depends on the order in which shared variables are accessed.

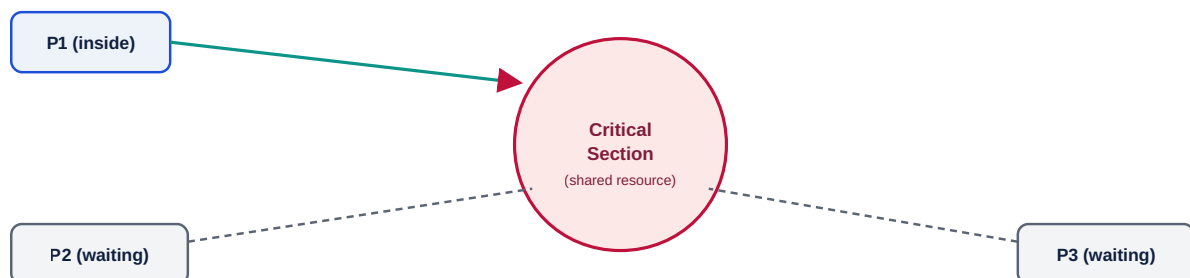


Fig 4 — Only P1 may execute in the critical section at a time; P2 and P3 must wait their turn.

Conditions a valid solution must satisfy

Mutual Exclusion

If P_i is in its critical section, no other process may enter its critical section.

Progress

The choice of which process enters next cannot be postponed indefinitely, and cannot be made by a process in its remainder section.

Bounded Waiting

There is a limit on how many times other processes may enter before a waiting request is granted.

Synchronization Tools

Semaphore — a protected variable used to lock a shared resource. Its value indicates the status of the resource.

Binary Semaphore

Takes only 0 or 1; used to implement mutual exclusion and synchronize concurrent processes.

Counting Semaphore

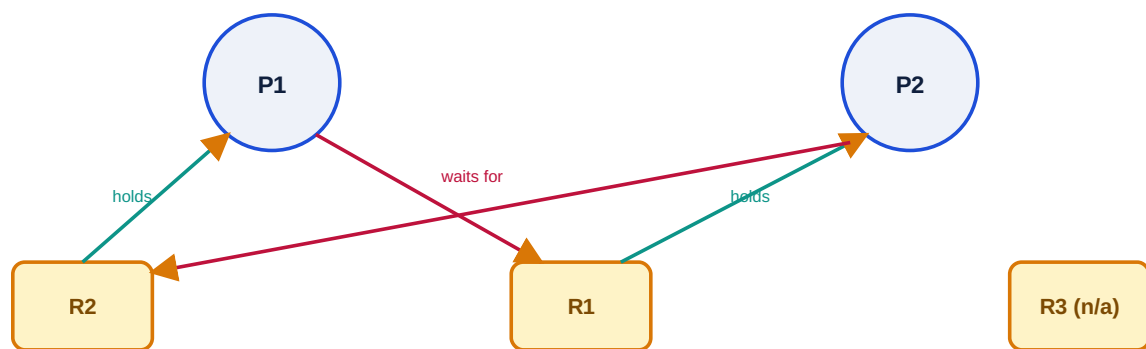
An integer variable whose value can range over an unrestricted domain.

Mutex — provides mutual exclusion: only the holder of the mutex (lock) may access the shared buffer at a time. While the producer fills the buffer, the consumer waits, and vice versa. A semaphore generalizes this concept.

05 Deadlocks

IMPORTANT

A deadlock occurs when a set of processes are blocked because each is holding a resource and waiting for another resource held by another process in the set. Deadlock requires four conditions to hold simultaneously.



P2 waits for R2 → circular wait completes back to P1

Fig 5 — Circular wait: P1 holds R2 and needs R1; P2 holds R1 and needs R2. Neither can proceed.

1. Mutual Exclusion

At least one resource is non-sharable — only one process can use it at a time.

2. Hold and Wait

A process holds at least one resource while waiting for additional resources.

3. No Preemption

A resource cannot be forcibly taken away; it is released only voluntarily.

4. Circular Wait

A set of processes wait for each other's resources in a circular chain.

Handling a Deadlock

1. **Prevention / Avoidance** — design the system so it never enters a deadlock state.
2. **Detection & Recovery** — let deadlock occur, then preempt resources to break it.
3. **Ignore the problem** — if deadlock is rare, let it happen and reboot. Both Windows and UNIX take this approach.

BANKER'S ALGORITHM

A deadlock-avoidance method modeled on banking: a bank never allocates cash in a way that would leave it unable to satisfy the maximum needs of all its customers.

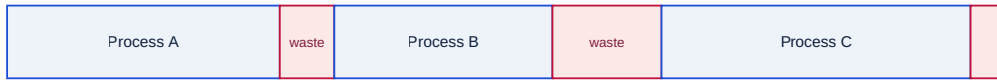
06 Memory Management

Memory management techniques allow memory to be shared among multiple processes.

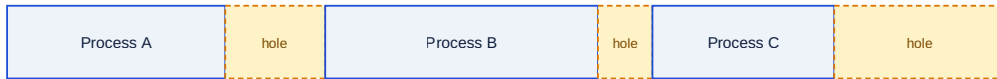
- **Overlays** — memory holds only the instructions and data needed at a given moment.
- **Swapping** — in multiprogramming, processes whose time slice has expired are swapped out of memory.

Partition Allocation Techniques

Fixed Partition — Internal Fragmentation



Variable Partition — External Fragmentation



Scattered free "holes" are too small individually to fit a new process — this is external fragmentation. Because contiguous allocation causes this, paging is used instead (see next page).

Fig 6 — Internal fragmentation (fixed partitions) vs. external fragmentation (variable partitions).

Variable Partition Allocation Schemes

Scheme	Rule
First Fit	Allot the first hole that is large enough to fit the process.
Best Fit	Allot the hole that leaves the minimum leftover space.
Worst Fit	Allot the hole that leaves the maximum leftover space.

NOTE

Best Fit does not necessarily give the best overall results for memory allocation.

Paging

Physical memory is divided into equal-sized **frames**; logical memory is divided into equal-sized **pages** of the same size. A page table maps each logical page to a physical frame.

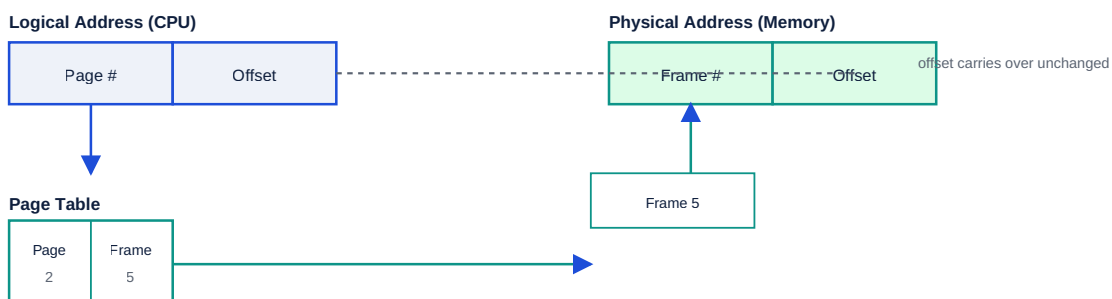


Fig 7 — Address translation: the page number is looked up in the page table to find the frame; the offset stays the same.

Segmentation gives users a logical view of memory, where the address space is a collection of variable-sized *segments* (e.g. code, stack, heap). Segmentation can be implemented with or without paging.

PAGE FAULT

A page fault is a hardware interrupt raised when a running program accesses a page that is part of its virtual address space but is not currently loaded into physical memory.

07 Page Replacement Algorithms IMPORTANT

1. First In First Out (FIFO)

The OS tracks pages in a queue; the oldest page (front of queue) is replaced first. Reference string: **1, 3, 0, 3, 5, 6** with 3 frames.

Reference	1	3	0	3	5	6
Frame 1	1	1	1	1	5	5
Frame 2		3	3	3	3	6
Frame 3			0	0	0	0
Result	Fault	Fault	Fault	Hit	Fault	Fault

Total: **5 page faults** (0 faults when 3 repeats, since it's already resident).

BELADY'S ANOMALY

FIFO can produce *more* page faults when the number of frames increases. For reference string 3 2 1 0 3 2 4 3 2 1 0 4: 3 frames give 9 faults, but 4 frames give 10 faults.

2. Optimal Page Replacement

Replaces the page that will not be used for the longest time in the future. Reference string: **7 0 1 2 0 3 0 4 2 3 0 3 2** with 4 frames.

Reference	7	0	1	2	0	3	0	4	2	3	0	3	2
Result	F	F	F	F	H	F	H	F	H	H	H	H	H

Total: **6 page faults**. Optimal is a theoretical benchmark — impossible in practice since the OS cannot know future requests.

3. Least Recently Used (LRU)

Replaces the page that was used least recently. Same reference string: **7 0 1 2 0 3 0 4 2 3 0 3 2** with 4 frames.

Reference	7	0	1	2	0	3	0	4	2	3	0	3	2
Result	F	F	F	F	H	F	H	F	H	H	H	H	H

Total: **6 page faults** — in this example LRU happens to match Optimal exactly.

08 Disk Scheduling

Seek Time

Time to move the disk arm to the required track.

Rotational Latency

Time for the desired sector to rotate under the read/write head.

Transfer Time

Time to actually transfer the data — depends on rotation speed and data size.

Disk Access Time

Seek Time + Rotational Latency + Transfer Time.

Algorithms

Algorithm	Behaviour
FCFS	Requests are serviced strictly in arrival order.
SSTF	The request nearest the current arm position is serviced first.
SCAN	The arm sweeps in one direction servicing requests, then reverses at the end — "elevator" algorithm.
C-SCAN	Like SCAN, but after reaching the end it jumps back to the start without servicing on the return sweep.
LOOK	Like SCAN, but reverses at the last request rather than travelling to the disk's physical end.
C-LOOK	Like C-SCAN, but jumps only to the last request at the other end, not the physical end.

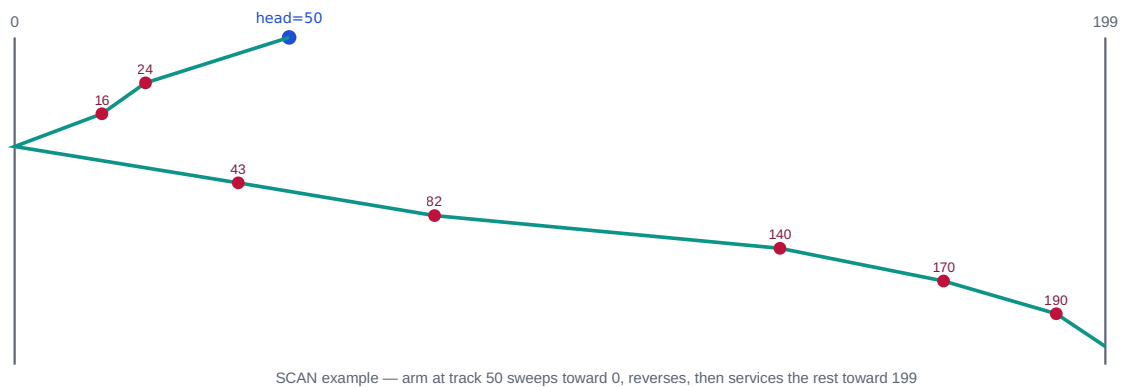


Fig 8 — SCAN (elevator) disk scheduling: the arm moves in one direction, reverses at the end, and continues.

09 Key Terms

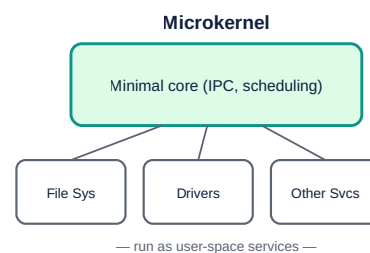
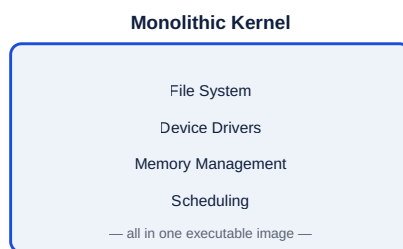


Fig 9 — A monolithic kernel bundles all OS services into one image; a microkernel keeps only essentials in the core and runs the rest as separate services (a macro kernel mixes both approaches).

Real-Time System

Used where rigid, well-defined time constraints are placed on processor operation.

Re-entrancy

Lets multiple users share one copy of a program's code at the same time — the code cannot modify itself, and each user's local data is stored separately.

Demand Paging

Memory not currently in use is swapped to disk to make room for an application's needs.

Virtual Memory

Lets processes execute even when they don't fully fit in physical memory. **IMP**

RAID

Redundant Array of Independent Disks — stores data redundantly to improve performance; there are 7 RAID levels.

Logical vs Physical Address

Logical address is generated by the CPU; physical address is what the memory unit actually sees.

Spooling

Data from multiple applications is queued to a disk file before being sent to a device such as a printer.

Starvation & Aging

Starvation: a waiting process never gets resources because they keep going to others. Aging: gradually raising a waiting process's priority to prevent starvation.

Thrashing

The processor spends most of its time swapping pages instead of executing instructions.

Fragmentation

Internal: wasted space inside fixed-size allocation units. **External:** wasted space between variable-size allocation units.

Advantages of Multithreaded Programming

1. Enhances responsiveness to the user.
2. Enables resource sharing within a process.
3. Economical compared to creating new processes.
4. Fully utilizes multiprocessor architectures.